Webes adatkezelés

XML-DOM

Kovács László, 2025

DOM

Az XML tartalom kezelésénél a maximális rugalmasságot az általános programozási nyelvek tudják biztosítani. Emiatt létrejöttek az XML API keretrendszerek is. Ebből a családból a legismertebb a DOM API.

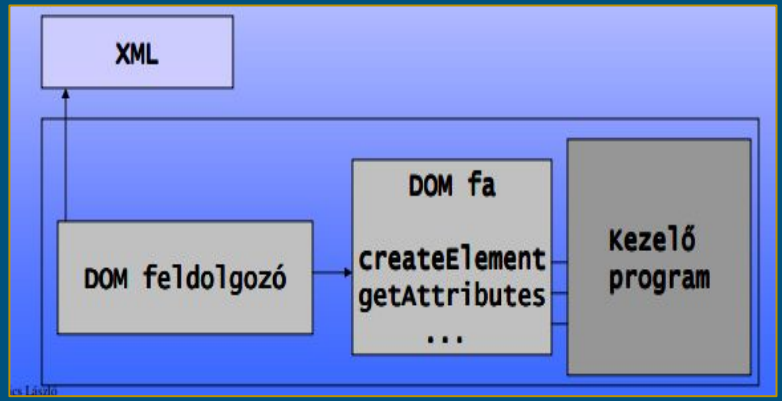
DOM API működési modell:

- a motor átoolvass az XML fát, majd felépít belőle egy saját objektum fát a memóriában.
- a gazdaalkalmazás szabadon dolgozhat a fával (CRUD műveletek a csomópontokra)
- OOP alapú XML kezelés, az XDM modellre épül
- rugalmas XML kezelés, elemi szintű műveletekre épül
- több gazdanyelvet is támogat (Java, C#, Python,..)

DOM

DOM API jellemzése:

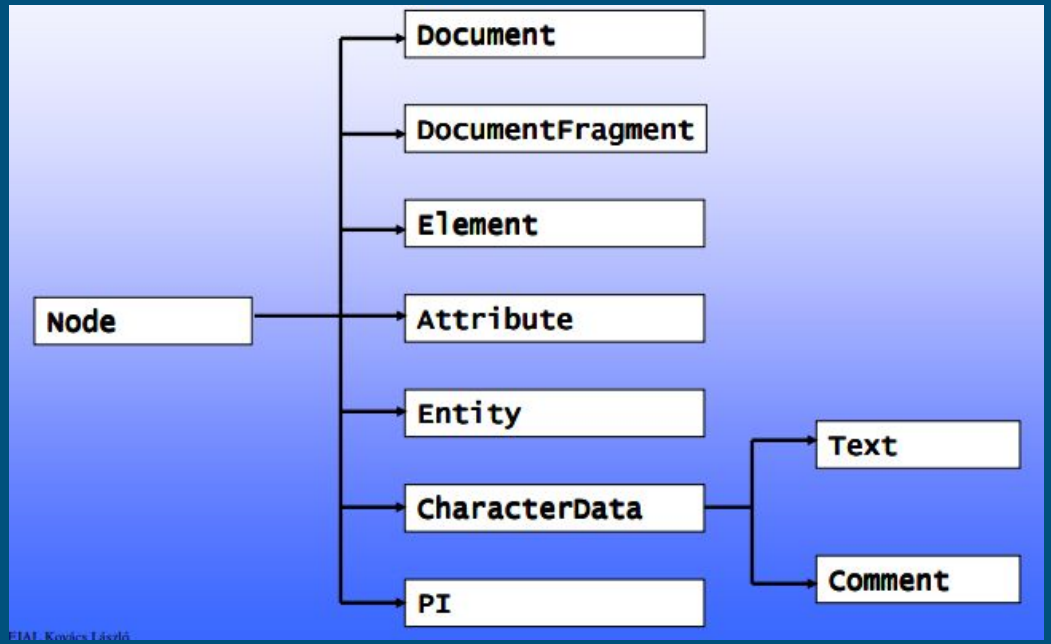
- Mind az olvasási, mind a módosítási műveleteket támogatja
- Navigációs műveleteken alapszik
- Halmaz és elem szintű műveletek támogatása
- A memória DOM kimenthető, szerializálható file is
- Nagyobb memóriaigénnyel bír
- Van speciális HTML-DOM (lazább, extra HTML jellegű funkciók) és XML-DOM (szigorúbb, általános)
- API használat lépései:
    - dokumentum olvasó létrehozása
    - XML beolvasás, DOM fa felépítése
    - DOM fa egyik elemének kiválasztása
    - elem szintű műveletek és navigávció
    - DOM fa kimentése

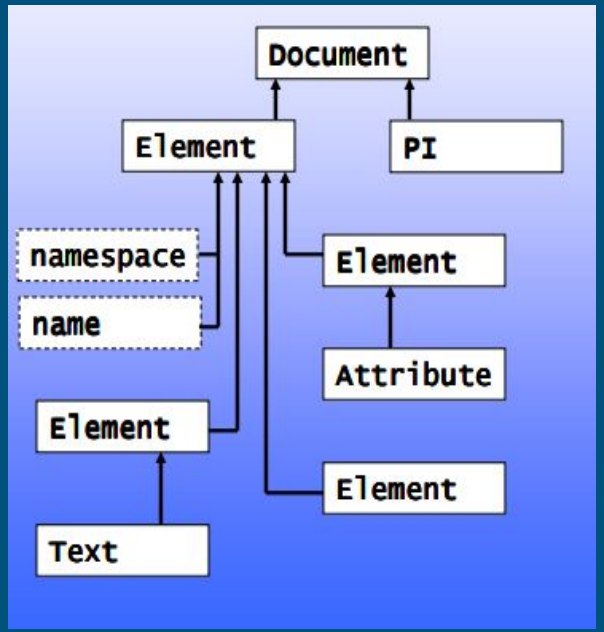


DOM

DOM struktúra modell

- XDM alapú, csomópont osztályok:





DOM

DOM fa felépítése (Java)

- A DOM fa felépítését egy DocumentBuilder osztály végzi
- A feldolgozás során be lehet állítani többek között a sémekezelés módját is
- A DOM fa felépítés lépései:

```
DocumentBuilderFactory dbf =
        DocumentBuilderFactory.newInstance()

dbf.setValidating(true);

DocumentBuilder db = dbf.newDocumentBuilder();

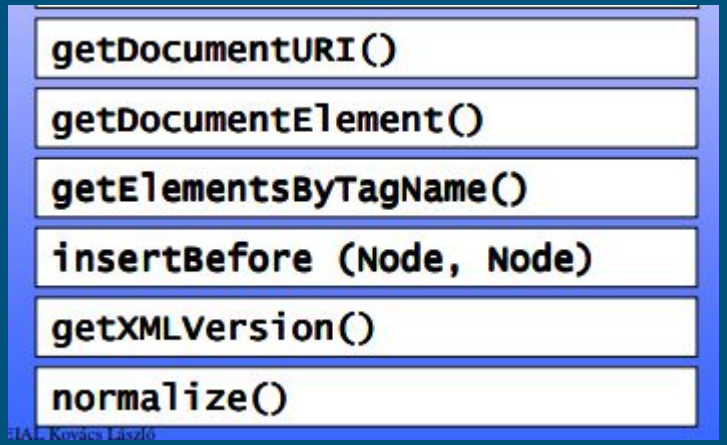
Document dom = db.parse (new File(args));

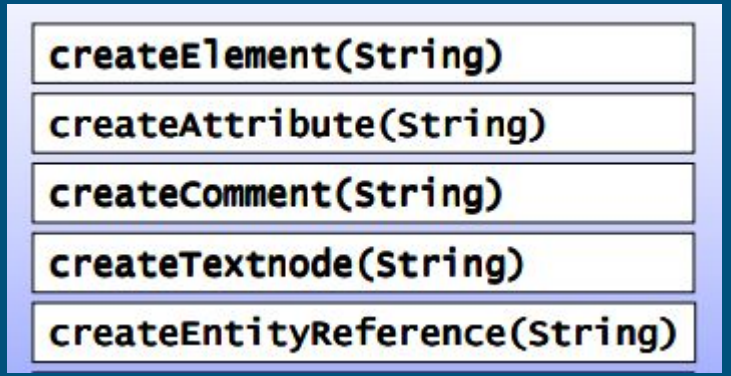
kezelo(dom);
```

DOM

Document osztály

- A teljes DOM fát érintő műveletek elvégzésre szolgál
- Belépési pontokat biztosít a DOM fába
- Új csomópontok létrehozatala is ezen keresztül történik
- XML beolvasás, kiírás biztosítása
- főbb metódusok:

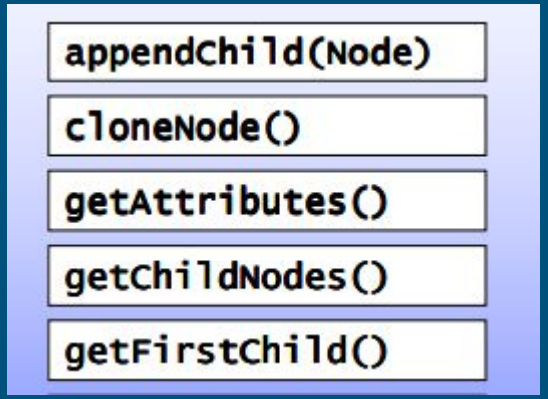




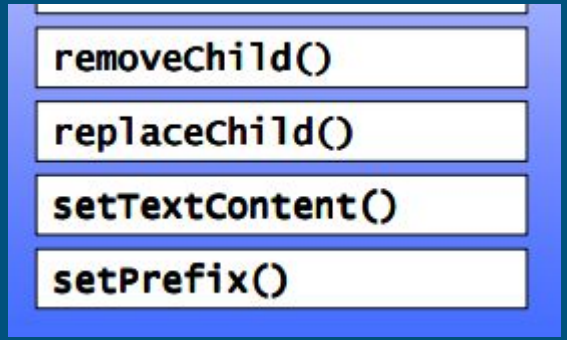
DOM

Node osztály

- Általános csomópont kezelő osztály
- A csomópont manipulációs lehetőségeit biztosítja
- Olvasási műveletek támogatása (név, típus, jellemzők, gyerekek)
- Módosítási műveletek támogatása (jellemzők átírása, gyerek hozzáadás vagy törlés)
- főbb metódusok:







DOM

Műveletek áttekintése

- navigációs műveletek:
  - e.parentNode() : szülő elem
  - e.childNodes() : gyerekelemek
  - e.firstChild(): első gyerekelem
  - e.nextSibling(): következő testvér
  - fa.getElementByTagName(“név”) : név alapjánelembegyűjtés
  - fa.getElementById(“id”) : id alapján elembegyűjtés
- lekérdezési műveletek:
  - e.getAttribute(“név”) : attributum lekérdezés
  - e.textContent() : szövegtartalom

DOM

Műveletek áttekintése

- módosítási műveletek:
  - fa.createElement(“név”) : új elem létrehozatala
  - e.appendChild(e) : gyerekelem hozzáadása
  - e.removeChild(e): gyerekelem törlése
  - e.setAttribute(“név”, érték) : attribútum beállítása

DOM objektum létrehozása (java)

```
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import org.w3c.dom.*;
import org.xml.sax.SAXException;

main() { …
     DocumentBuilderFactory dbf =
          DocumentBuilderFactory.newInstance();
     try {
          DocumentBuilder db = dbf.newDocumentBuilder();
          Document dom = db.parse(new File(fnev));
           query1(dom);
     }catch(ParserConfigurationException pce) { }
     catch(SAXException se) { }
     catch(IOException ioe) {  }
```

Gyökérelem lekérdezése:

```
Document dok;
Node root = dok.getDocumentElement();
```

Gyerekelemek bejárása:

```
Node elem, gyerek;
for (p=0; p<elem.getChildNodes().getLength(); p++) {
      gyerek = elem.getChildNodes().item(p);
       …
}
```

Elem nevének és tipusának lekérdezése:

```
Node elem;
if (elem.getNodeType()==Node.ELEMENT_NODE) {
      if (elem.getNodeName().compareTo("ar") == 0) {}
         …
}
```

Új elem felvitele

```
Document dok;
Node ujelem, elem;
Node ujelem = dok.createElement("konyv");
elem.appendChild(ujelem);
ujelem.setTextContent("Babits");
```

Elem törlése

```
Node elem, gyerek;
for (p=0; p<elem.getChildNodes().getLength(); p++) {
            gyerek = elem.getChildNodes().item(p);
      elem.removeChild(gyerek);
}
```

Elemjellemzők bejárása:

```
Node elem;
for (p=0; p<elem.getAttributes().getLength();p++) {
        Node elemuj = elem.getAttributes().item(p);
         …
}
```

Szövegérték növelése:

```
Node elem;
ear = Integer.parseInt(elem.getTextContent());
String ujszov = String.valueOf(ear+nov);
elem.setNodeValue(ujszov);
```

XML dokumentum validálása

A feldolgozó opcionálisan elvégzi a bejövő dokumentum validálását is a kapcsolódó séma alapján

```
DocumentBuilderFactory dbf =
        DocumentBuilderFactory.newInstance();
dbf.setValidating(true);

DocumentBuilder db = dbf.newDocumentBuilder();
db.setErrorHandler(new hibakez());

Document dom = db.parse(new File(fnev));
```

Az XML dokumentum gyökérelemében megadott sémára ellenőriz

XML dokumentum kiírása állományba

```
String fnev2 =
     "c:\\users\\kovacs_l\\jegyzetxml\\proba_ki.xml";

Document dom = db.parse(new File(fnev));

OutputFormat f = new OutputFormat(dom);

XMLSerializer outp = new XMLSerializer
    ( new FileOutputStream (new File(fnev2)) , f);

outp.serialize(dom);
```

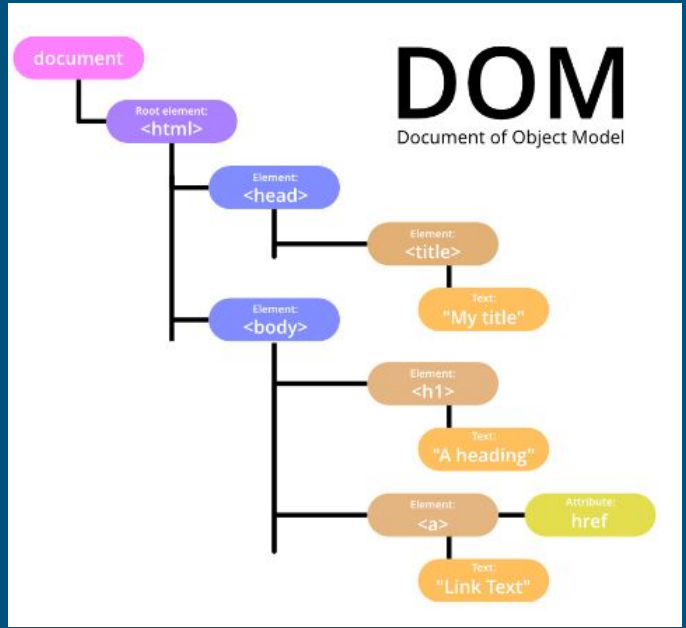
DOM

Python DOM API

- az xml.dom library gazdag funkcionalitású DOM kezelő
- betöltése:

```
import xml.dom
```

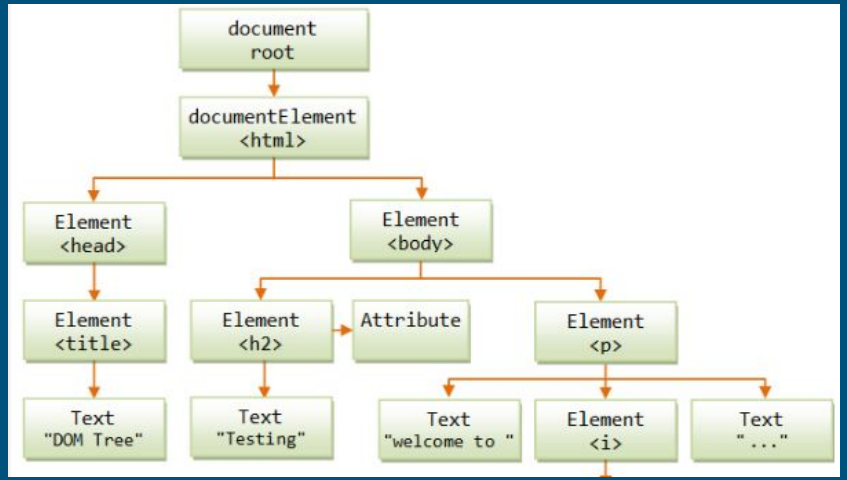
- főbb osztályok:
  - Document: teljes dokumentum
  - Node: általános csomópont
  - NodeList: csomópont lista
  - Element: jelölő elem
  - Attr: attribútum
  - Text: szöveg csomópont
  - Comment: megjegyzés
  - ProcessingInstruction: PI



DOM

Python DOM Document class

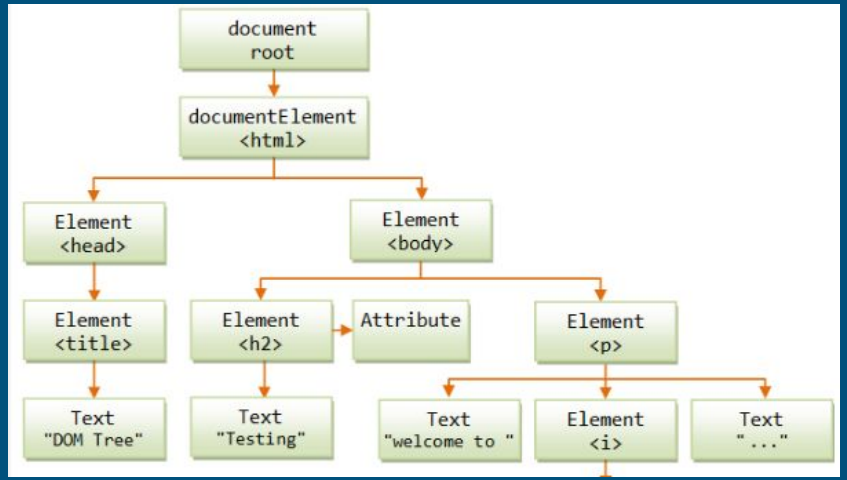
- a DOM fa főbb metódusai:
  - documentElement: a gyökérelem
  - getElementsByTagName(név): adott nevű elemek begyűjtése
  - createElement(név): új elem létrehozatala, nincs még a fában
  - createAttribute(név): új attribútum létrehozatala, nincs még a fában



DOM

Python DOM Node class

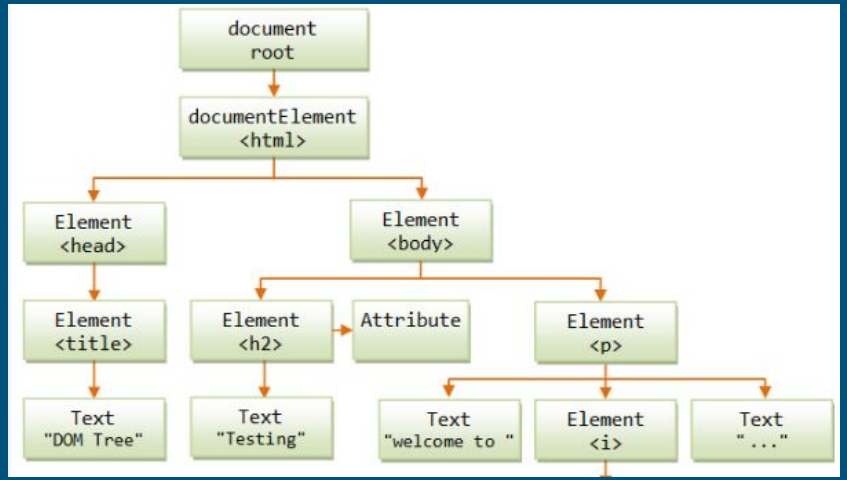
- a csomópont főbb adattagjai:
  - nodeType : csomópont típusa (XDM)
  - parentNode : szülő csomópont
  - attributes : attribútumok
  - childNodes : gyerekcsomópontok
  - localName: rövid neve
  - nodeValue: szöveges értéke
- a csomópont főbb metódusa:
  - appendChild(elem):új gyerek
  - removeChild(elem): gyerek törlése
  - replaceChild(ujelem,relem):
    gyerekelem cseréje



DOM

Python DOM Element class

- a DOM elem főbb metódusai:
    - getElementsByTagName(név) : adott nevű leszármazottak kigyűjtése
    - getAttribute(név) : attribútum értékének lekérdezése
    - setAttribute(név, érték) : attribútum beállítása
    - removeAttribute(név) : attribútum törlése



```
from xml.dom import getDOMImplementation

# Get the DOM Implementation
impl = getDOMImplementation()

# Create a new XML document
doc = impl.createDocument(None, "library", None)

# Get the root element
root = doc.documentElement

# Create a book element
book = doc.createElement('book')
book.setAttribute('category', 'fiction')
root.appendChild(book)
```

```
# Create a title element and add text
title = doc.createElement('title')
title_text = doc.createTextNode('The Great Gatsby')
title.appendChild(title_text)
book.appendChild(title)

# Serialize the document (converting to a string)
from xml.dom.ext import PrettyPrint
from io import StringIO

output = StringIO()
PrettyPrint(doc, output)
print(output.getvalue())
```

```
# Accessing attributes
book = root.getElementsByTagName('book')[0]
category = book.getAttribute('category')
print(category)  # Output: children

# Accessing text content
title = book.getElementsByTagName('title')[0]
print(title.firstChild.nodeValue)  # Output: Harry Potter

# Accessing sibling nodes
price = book.getElementsByTagName('price')[0]
print(price.previousSibling.previousSibling.tagName)  # Output: year
```

```
# Modify the price
book = root.getElementsByTagName('book')[0]
price = book.getElementsByTagName('price')[0]
price.firstChild.nodeValue = '39.99'

# Add a new element
publisher = doc.createElement('publisher')
publisher.appendChild(doc.createTextNode('Bloomsbury'))
book.appendChild(publisher)

# Serialize the modified document
output = StringIO()
PrettyPrint(doc, output)
print(output.getvalue())
```

```
# Remove the year element
year = book.getElementsByTagName('year')[0]
book.removeChild(year)

# Serialize and print the modified XML document
output = StringIO()
PrettyPrint(doc, output)
print(output.getvalue())
```

Colab::oktatas.ipynb